

Project Report: Parking System

Title: Automated Parking Management System

Group Members :

Faryal Asif (030)

Maryam Tariq (062)

Warda Nayab (091)

Hamna Qandeel (038)

Hamna Noor (037)

(III – B)

Submitted to: Mam Qurat ul Ain

Artificial Intelligence

Introduction

Parking management is a crucial aspect of urban planning, particularly in cities where the demand for parking often exceeds supply. An efficient parking system not only saves time but also optimizes the use of available resources. This report presents a project to develop an automated parking management system that allows users to check, reserve, and free parking slots through a simple graphical user interface (GUI). Additionally, the system includes an AI-driven feature to suggest the next available parking slot.

The purpose of this project is to simplify the parking process for users while ensuring that parking resources are utilized effectively. This system, implemented using Python and Tkinter, provides a scalable solution for small to medium-sized parking lots.

Technologies Used

1. Python Programming Language

Python was chosen for its simplicity, readability, and vast library support. Python's Tkinter library was used for developing the graphical user interface (GUI). Additionally, Python's robust data structures were employed for managing parking slot data efficiently.

2. Tkinter

Tkinter is Python's standard GUI library. It was used to create the user interface, which includes input fields, buttons, and status displays. Tkinter enables rapid development of user-friendly GUIs.

3. Random Module

The random module was used for generating AI-based suggestions for the next available parking slot. This module helps simulate simple AI logic by selecting a free slot randomly from the list of available slots.

Language Model

The Parking Management System is based on Python, a high-level programming language known for its clean syntax and versatility. Python's object-oriented features were leveraged to design the system in a modular and maintainable way. The system is built around the `ParkingSystem` class, which encapsulates all functionalities related to parking slot management.

The GUI components, created using Tkinter, interact with this class to provide a seamless user experience.

How the System Works

1. Initialization

The system initializes with 50 parking slots, all marked as free. These slots are represented as a dictionary, where keys are slot IDs (1 to 50) and values are Boolean flags indicating whether the slot is reserved (`True`) or free (`False`).

2. Slot Checking

Users can enter a slot ID (1 to 50) and check its status. The system validates the input and displays whether the slot is free or reserved.

3. Reserving a Slot

Users can reserve a parking slot by entering its ID. The system ensures that:

- The slot ID is valid.
- The slot is currently free.
- The user has not already reserved another slot. If these conditions are met, the system marks the slot as reserved and updates the status.

4. Freeing a Slot

Users can free a reserved slot by entering its ID. The system validates the input and checks whether the slot is currently reserved. If valid, the system marks the slot as free and updates the status.

5. AI-Based Slot Suggestion

The system provides an AI-based suggestion for the next available slot. It scans the list of free slots and randomly selects one to suggest to the user. This feature is particularly useful for guiding users to an optimal parking slot quickly.

6. Graphical User Interface (GUI)

The GUI is the primary medium for user interaction. It consists of:

- An input field for entering the slot ID.
- Buttons for checking, reserving, freeing, and suggesting slots.

- A status label to display messages and feedback. The GUI ensures that users can interact with the system intuitively.
-

Detailed Implementation

1. The ParkingSystem Class

The `ParkingSystem` class contains the core logic of the system. It includes methods for:

- Checking slot status (`check_slot`).
- Reserving a slot (`reserve_slot`).
- Freeing a slot (`free_slot`).
- Suggesting the next available slot (`suggest_next_slot`).

2. The ParkingApp Class

The `ParkingApp` class manages the GUI. It interacts with the `ParkingSystem` class to perform operations and update the interface accordingly. Key components of the GUI include:

- Labels for instructions and status updates.
- Entry widgets for user input.
- Buttons for triggering system functions.

3. Event Handling

Each button in the GUI is linked to a corresponding method in the `ParkingApp` class. These methods validate user input, call the appropriate methods from the `ParkingSystem` class, and display the results.

4. Randomized AI Suggestions

The `suggest_next_slot` method uses Python's `random.choice` function to select a free slot from the list of available slots. If no slots are available, the system informs the user.

Graphical User Interface

The GUI is designed for ease of use. Here is a walkthrough of its components:

1. **Input Field:** Users can enter the slot ID they want to check, reserve, or free.
2. **Buttons:**
 - o **Check Slot:** Checks the status of a slot.
 - o **Reserve Slot:** Reserves the specified slot if available.

- o **Free Slot:** Frees the specified slot if reserved.
 - o **Suggest Next Slot:** Provides an AI-driven suggestion for the next available slot.
 - 3. **Status Label:** Displays messages and feedback based on user actions.
-

Example Scenarios

1. **Scenario 1: Checking a Slot**
 - o User enters 25 and clicks "Check Slot."
 - o The system responds with "Slot 25 is FREE" if the slot is not reserved.
 2. **Scenario 2: Reserving a Slot**
 - o User enters 12 and clicks "Reserve Slot."
 - o The system responds with "Slot 12 has been RESERVED" if successful.
 3. **Scenario 3: Freeing a Slot**
 - o User enters 12 and clicks "Free Slot."
 - o The system responds with "Slot 12 has been FREED" if successful.
 4. **Scenario 4: AI Suggestion**
 - o User clicks "Suggest Next Slot."
 - o The system responds with "AI suggests parking slot: 18" if slot 18 is available.
-

Conclusion

The Automated Parking Management System offers a user-friendly and efficient solution for managing parking spaces. By combining the simplicity of Python with the capabilities of Tkinter, this project demonstrates how software can optimize real-world processes. The inclusion of an AI-based suggestion feature adds value by providing intelligent guidance to users.

Future enhancements could include:

- Expanding the system to manage multiple parking lots.
- Adding a database for persistent storage of parking data.
- Integrating sensors for real-time slot availability updates.

This project is a testament to how programming can solve everyday problems and improve operational efficiency in urban settings.